

Corso di Calcolo Parallelo
Laurea Magistrale in Informatica—Università di Bologna
Demo Prova Scritta

- La prova dura 90 minuti
- Durante la prova non è consentito consultare libri, appunti o altro materiale.
- Non è consentito l'uso di dispositivi elettronici (ad esempio, cellulari, tablet...), né interagire in alcun modo con gli altri ammessi alla prova pena l'esclusione, che potrà avvenire anche dopo il termine della stessa.
- Le risposte devono essere scritte **a penna** su questi fogli, in modo **leggibile**. Le parti illeggibili o scritte a matita saranno ignorate.
- Eventuali altri fogli possono essere utilizzati per la brutta copia, vanno consegnati ma non verranno valutati.
- Si risponda in modo sintetico ma il più possibile esaustivo.
- I voti saranno pubblicati su AlmaEsami.
- I voti della prova scritta e del progetto restano validi fino alla sessione d'esame di **febbraio 2027** inclusa. Dopo tale data tutti i voti in sospeso saranno persi.

NON SCRIVERE NELLA TABELLA SOTTOSTANTE

D. 1	D. 2	D. 3	D. 4	D. 5
/ 6	/ 6	/ 6	/ 6	/ 6

I fogli verranno stampati su un'unica facciata, quindi sarà disponibile anche il retro di ciascuno per le risposte. Verranno forniti fogli protocolli per la brutta copia; tali fogli dovranno essere consegnati al termine dell'esame, assieme al testo.

Domanda 1.

Quali considerazioni è possibile fare per individuare il livello di granularità “ideale” di decomposizione del dominio per una data applicazione parallela? Discutere.

Risposta.

Domanda 2.

Si consideri il seguente frammento di codice che risolve un sistema lineare $Ax = b$, dove $A[] []$ è una matrice quadrata di dimensioni $N \times N$ in forma triangolare superiore, $x[]$ e $b[]$ sono due array di lunghezza N .

```
#define N ...
double A[N][N] = ...;
double b[N] = ...;
double x[N];
for (int i = N-1; i >= 0; i--) { /* (1) */
    x[i] = b[i];
    for (int j = i+1; j < N; j++) { /* (2) */
        x[i] += -A[i][j] * x[j];
    }
    x[i] /= A[i][i];
}
```

Ad esempio, un sistema lineare in forma triangolare superiore è il seguente:

$$\begin{array}{rcl} 3x[0] + 2x[1] - 7x[2] & = & 13 \\ -7x[1] + 12x[2] & = & -31 \\ 42x[2] & = & 18 \end{array}$$

Il codice sopra risolve il sistema lineare a ritroso, calcolando prima il valore di $x[N-1]$ usando l'ultima equazione, poi $x[N-2]$ usando la penultima (sfruttando il valore $x[N-1]$ calcolato al passo precedente), e così via. (Non è necessario conoscere i dettagli dell'algoritmo per svolgere questo esercizio; è sufficiente esaminare il codice fornito).

Rispondere alle seguenti due domande:

1. È possibile parallelizzare il ciclo “for” esterno (quello marcato con il commento (1)) utilizzando costrutti OpenMP? Giustificare la risposta.
2. È possibile parallelizzare il ciclo “for” interno (quello marcato con il commento (2)) utilizzando costrutti OpenMP? Giustificare la risposta.

Si assuma di non poter modificare il codice, ma solo aggiungere uno o più costrutti OpenMP con eventuali clausole.

Risposta.

Domanda 3.

Descrivere un algoritmo per architetture a memoria distribuita basato su MPI che, dato un array $v[N]$ di lunghezza N , determina se $(v[i] == i)$ per ogni indice i oppure no. Detto in altri termini, il programma deve controllare se l'array $v[]$ contiene i valori $\{0, 1, 2, \dots, N-1\}$, in quest'ordine, oppure no.

Si assuma che:

- la lunghezza N dell'array sia molto maggiore del numero P di processi MPI;
- la lunghezza N dell'array sia multipla di P ;
- la lunghezza N dell'array sia inizialmente nota a tutti i processi.
- l'array $v[]$ sia inizialmente presente solo nella memoria del processo 0 (il master);
- al termine dell'esecuzione solo il processo 0 debba stampare un opportuno messaggio che indichi se l'array $v[]$ è un array enumerativo oppure no.

Si descriva nel modo più preciso possibile il comportamento di ciascun processo MPI. **Non è richiesto scrivere codice:** è possibile usare pseudocodice e/o linguaggio naturale, purché la spiegazione sia il più precisa possibile. In particolare, si indichi quale/i funzioni MPI si userebbero nei vari passi, e quali buffer di memoria è necessario usare/allocare.

Risposta.

Domanda 4.

1. Illustrare brevemente che cosa è la *shared memory* presente nelle GPU CUDA.
2. Descrivere una applicazione in cui l'uso della shared memory **produca** un significativo miglioramento delle prestazioni rispetto ad un analogo programma che non ne fa uso, motivando la risposta.
3. Descrivere una applicazione in cui l'uso della shared memory **non produca** un significativo miglioramento delle prestazioni rispetto ad un analogo programma che non ne fa uso, motivando la risposta.

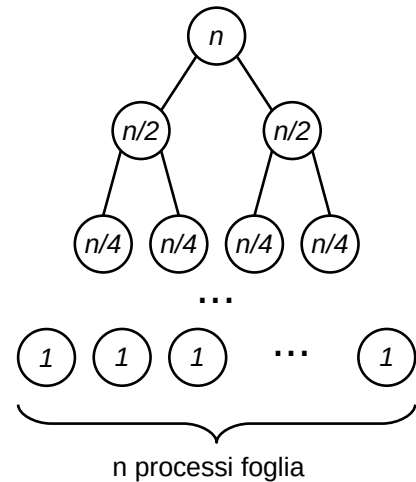
(È possibile utilizzare esempi visti a lezione o in laboratorio, oppure proporre di nuovi; in ogni caso è necessario descrivere l'applicazione con un livello di dettaglio adeguato).

Risposta.

Domanda 5.

Si consideri la seguente implementazione OpenMP dell'algoritmo di ordinamento Merge Sort basata sui task; per semplicità si utilizza la notazione $v[i..j]$ per indicare la porzione di un array $v[]$ compresa tra gli indici i e j , estremi inclusi.

```
mergesort(v[i..j])
{
    if (i >= j)
        return;
    else {
        m = (i+j)/2;
        #pragma omp task shared(v, i, m, tmp)
        mergesort(v[i..m]);
        #pragma omp task shared(v, j, m, tmp)
        mergesort(v[m+1..j]);
        #pragma omp taskwait
        merge(v[i..m], v[m+1..j]);
    }
}
```



La funzione viene inizialmente invocata da un singolo thread di una regione parallela con la chiamata `mergesort(v[0..n-1])`. La funzione `merge(v[i..m], v[m+1..j])` effettua la fusione ordinata di due sottovettori adiacenti e ordinati $v[i..m]$ e $v[m+1..j]$, memorizzando il risultato in $v[i..j]$. Il costo asintotico di `merge()` è proporzionale alla lunghezza di $v[i..j]$. Nel diagramma a destra, ogni nodo rappresenta un processo, e il valore indicato all'interno indica il lavoro (*work*) svolto da quel processo.

Determinare *work* e *span* della funzione `mergesort()` applicata ad un array di lunghezza n , supponendo di avere $p = n$ processori.

Risposta.

Qui verrà incollato il “cheat sheet”, che pertanto sarà sempre disponibile assieme al testo dell'esame